

Тестовое задание для Backend разработчика

Важно: Ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе».

Срок выполнения: 3 дня

Формат сдачи: ссылка на репозиторий

Бизнес кейс: AI-агент проверки целевого использования льготных кредитов

Проект: AI-агент для автоматизации проверки документов, планируемых к заявлению в банк для оплаты

Проблема: неструктурированный вход, отсутствие единого ID, ручная валидация, нет версионности, сложности с офлайн-работой, риски безопасности.

Как процесс работает сейчас (AS IS)

Сотрудники вручную собирают пакет документов (договор, спецификация, УПД/акт, счет), проверяют сроки, подписи, печати и соответствие условиям программы. Данные хранятся разрозненно (локальные папки, сетевые диски), нет единого идентификатора пакета, отсутствует версионность документов. Валидация данных происходит «на глаз», а история изменений не ведется. Это создает риски безопасности и сложности с аудитом.

Как процесс должен работать (TO BE):

- Специалист казначейства загружает пакет документов через веб-интерфейс или кладет в сетевую папку.
- Бэкенд сохраняет файлы в хранилище (файловое/БД), присваивает уникальный ID пакету и каждой сущности (договор, акт), фиксирует метаданные (дата загрузки, пользователь).
- Система валидирует структуру пакета: проверяет наличие обязательных файлов, форматы, целостность.
- Бэкенд предоставляет API для фронтенда и AI-модулей.
- AI-модуль осуществляет следующие функции: загрузка/получение файлов, запуск проверки документов, получение статуса проверки и отчета.
- Реализуется версионность: при повторной загрузке документа создается новая версия, сохраняется история изменений.
- Обеспечивается безопасность: разграничение прав доступа пользователей.

Ключевые сценарии использования (User Stories):

- Как специалист казначейства, я хочу загрузить пакет документов в систему, чтобы бэкенд сохранил их в защищенном хранилище и вернул мне уникальный идентификатор заявки.
- Как руководитель, я хочу видеть историю изменений по каждому пакету документов, чтобы контролировать корректность данных.
- Как разработчик AI-модуля, я хочу получить через API список файлов пакета и их содержимое для извлечения данных.

1. Тестовое задание

Задача: Реализовать REST API сервис проверки пакетов документов с сохранением истории в PostgreSQL.

- 1) Реализовать эндпоинт POST /api/checks: принимает файлы (multipart/form-data) и название программы (federal / regional), запускает проверку, возвращает результат и сохраняет его в БД.
- 2) Реализовать GET /api/checks — список всех проверок (id, дата, программа, статус, количество документов).
- 3) Реализовать GET /api/checks/{id} — полный результат конкретной проверки.
- 4) Определять тип документа по имени файла (договор, спецификация, счёт, акт/УПД) и проверять комплектность пакета: для federal обязательны договор, спецификация, счёт, акт; для regional — договор, счёт, акт. Анализ по количеству и по паттерну имени.

- 5) Добавить проверки: недопустимый формат файла (только PDF, DOCX, JPG, PNG), размер файла более 20 МБ, нераспознанное имя файла — предупреждение уровня warning.
- 6) Итоговый статус: approved (нет нарушений), rejected (есть хотя бы одна ошибка), check_in_progress.
- 7) Покрыть тестами через pytest логику определения типа документа и формирования итогового статуса — минимум 5 тест-кейсов.
- 8) Весь проект должен подниматься одной командой docker compose up — предоставь Dockerfile для сервиса и docker-compose.yml с сервисом PostgreSQL.
- 9) Репозиторий должен содержать README.md: как запустить, как запустить тесты, описание переменных окружения из .env.example.

Формат ответа `POST /api/checks`:

```
{
  "check_id": "abc123",
  "status": "reject",
  "status_label": "Нельзя заявлять в банк",
  "reason": "Отсутствует спецификация к договору.",
  "issues": [
    { "level": "error", "message": "Отсутствует обязательный документ: спецификация" },
    { "level": "warning", "message": "Не удалось определить тип документа: «scan_0041.jpg»" }
  ],
  "documents": [
    { "name": "договор_47.pdf", "detected_type": "contract", "size_kb": 142 }
  ],
  "extracted": {
    "contractor": "ООО «ТехАгро»",
    "amount": "1 250 000 ₽",
    "date": "01.03.2025",
    "subject": "Поставка минеральных удобрений"
  },
  "checked_at": "2025-03-15T14:32:00Z"
}
```

Требования к реализации: конфигурация через .env, корректные HTTP-статусы (400, 404, 422), валидация через Pydantic, миграции через Alembic.

Технические требования:

Стек : Python 3.11, FastAPI, PostgreSQL, SQLAlchemy.

Управление зависимостями — на выбор (pip + requirements.txt, uv, poetry и т.д.).

Что должно быть в README:

1. Как запустить проект, как запустить тесты.
2. Какие технологии использованы и почему.
3. Архитектуру решения.
4. Описание переменных окружения из .env.example..
5. Как запустить тесты.

2. Мотивация участия в проекте:

1. Почему тебе интересен этот проект?
2. Как ты видишь свою роль в команде, которая создаёт продукт?
3. Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?